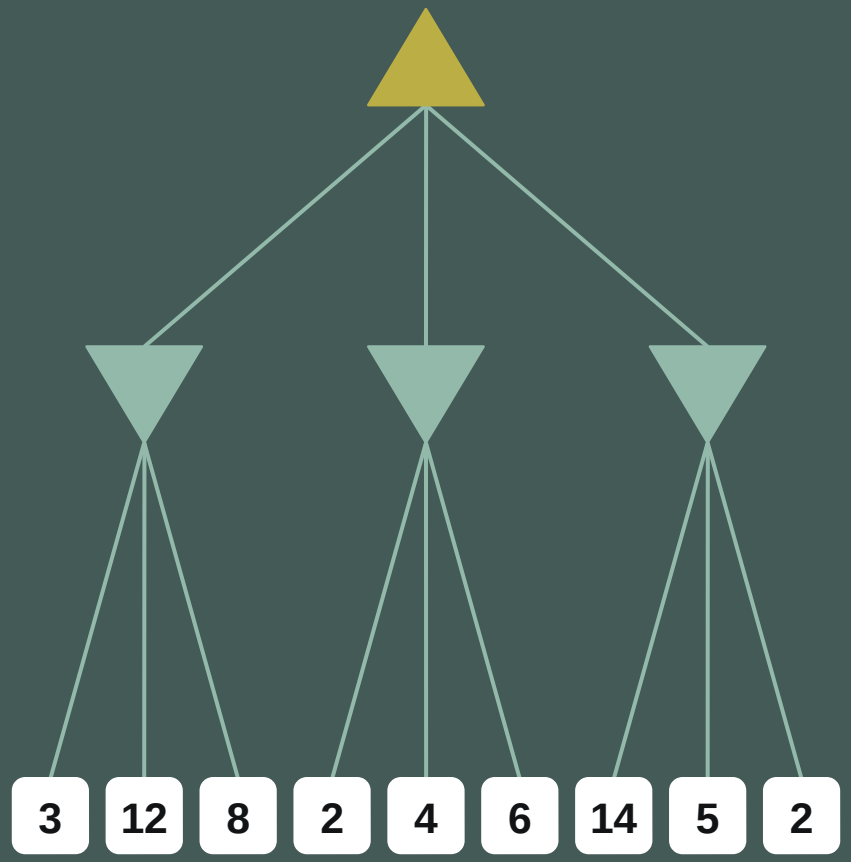
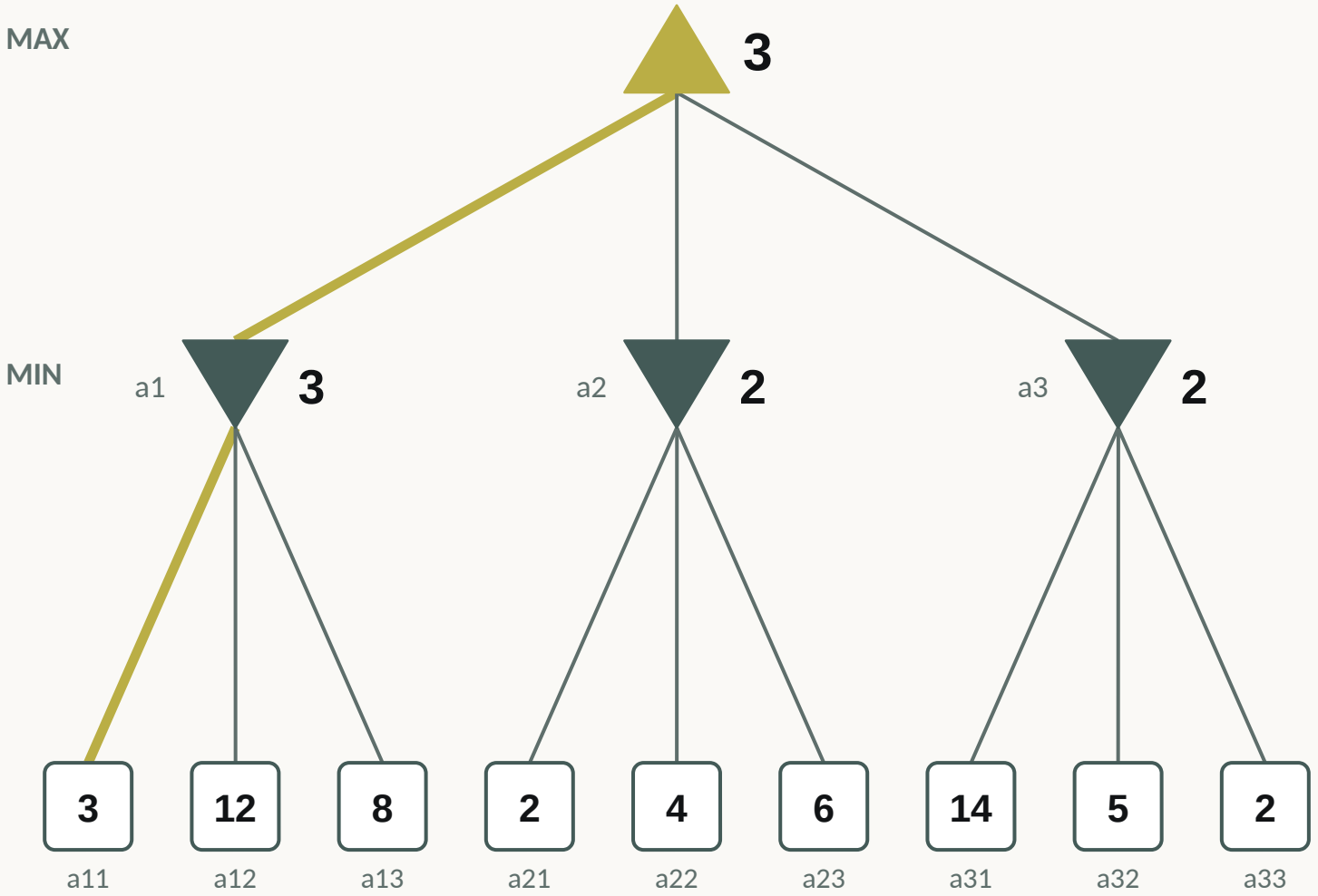
Adversarial search

Minimax and Alpha-Beta in Python

From the game tree to a running program



One tree, two algorithms



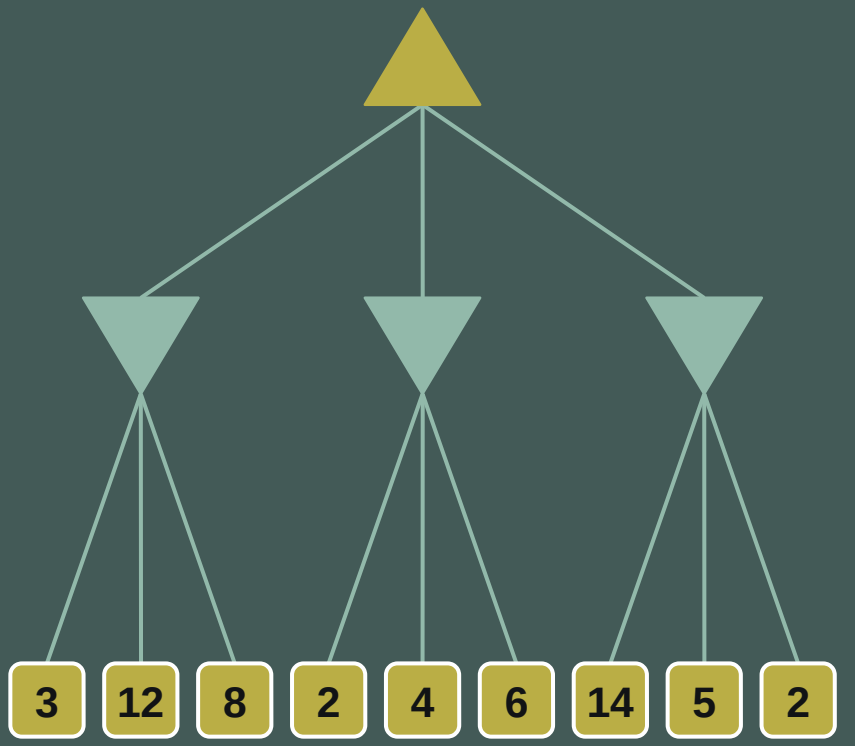
3

is what MAX can guarantee by playing a1: MIN will answer with a11.

We will store this tree in Python dictionaries and solve it twice: first with minimax, which looks at every leaf, then with alpha-beta, which skips the leaves that cannot change the decision.

Part 1: Minimax

Explore every leaf and back the values up the tree



The tree as Python data

```python
# The game tree
TREE = {
    "root": ["a1", "a2", "a3"],
    "a1": ["a11", "a12", "a13"],
    "a2": ["a21", "a22", "a23"],
    "a3": ["a31", "a32", "a33"],
}

# Utility of the leaves
UTILITY = {"a11": 3, "a12": 12, "a13": 8,
           "a21": 2, "a22": 4, "a23": 6,
           "a31": 14, "a32": 5, "a33": 2}

# Who moves (every other inner node is MIN)
MAX_TO_MOVE = {"root"}

def children(pos):
    return TREE.get(pos, [])
```

1. **TREE: the edges**

   Each key maps a node to the list of its children.

2. **UTILITY: the leaves**

   Only terminal positions have a utility. Here, gains for MAX.

3. **Whose turn it is**

   A set with the MAX nodes. Every other inner node is MIN.

4. **children()**

   Leaves are not keys of TREE, so they get an empty list.

minimax() in four steps

```
def leaf_value(pos):
    val = UTILITY[pos]
    print(f"  leaf {pos} = {val}")
    return val

def minimax(pos):
    """Return (value, best_next_pos)."""
1   kids = children(pos)
2   if not kids:                              # terminal
        return leaf_value(pos), None
3   pairs = [(minimax(k)[0], k) for k in kids]
4   if pos in MAX_TO_MOVE:
        return max(pairs, key=lambda p: p[0])
    return min(pairs, key=lambda p: p[0])
```

1. **Collect the children**

   children() gives the positions reachable with one move.

2. **Or stop at a leaf**

   With no children, return the utility and no next move.

3. **Evaluate each child**

   minimax calls itself on every child and keeps (value, move) pairs.

4. **Pick the best pair**

   MAX takes the largest value, MIN the smallest.

(value, move) pairs: one call picks both

```
pairs = [(3, "a1"), (2, "a2"), (2, "a3")]
max(pairs, key=lambda p: p[0])     # (3, 'a1')
min(pairs, key=lambda p: p[0])     # (2, 'a2')
```

At root, after evaluating the three MIN nodes:

(3, a1) **(2, a2)** **(2, a3)** → **value = 3**
move = a1

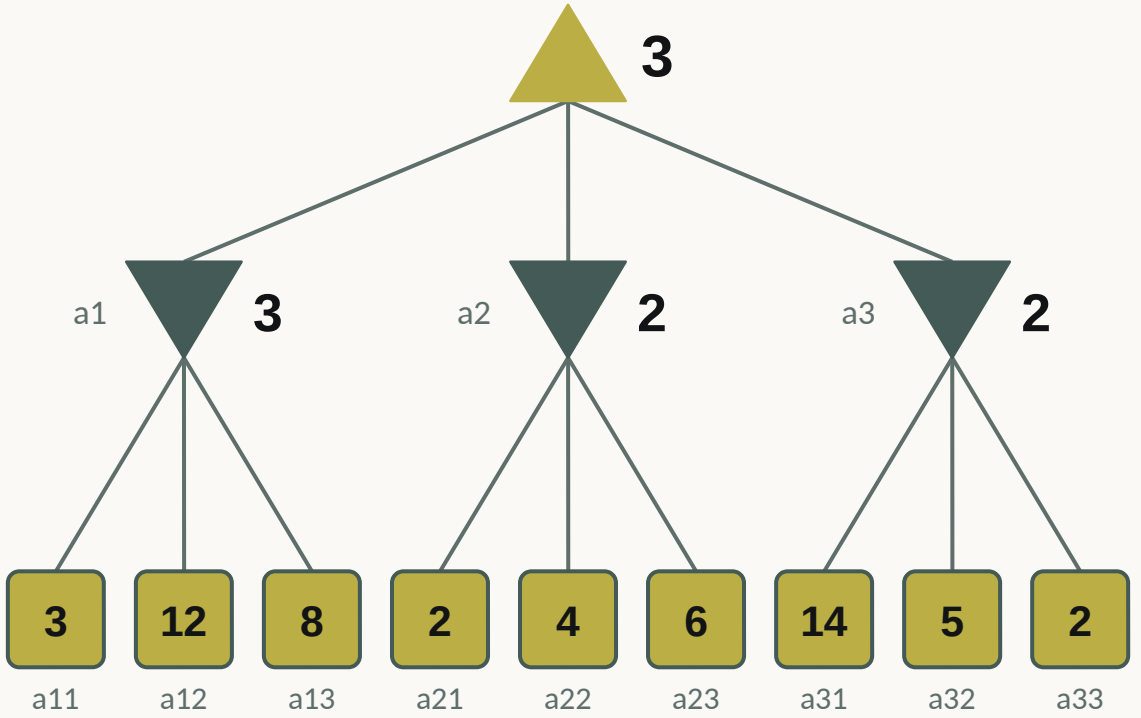
Keeping the move next to its value means that when we find the best value, we already know which move produced it. No second search is needed.

Why key=?

Without it, Python compares whole tuples and breaks ties by the move's name. With key=lambda p: p[0] only the value counts, and a tie keeps the first move found.

Running minimax

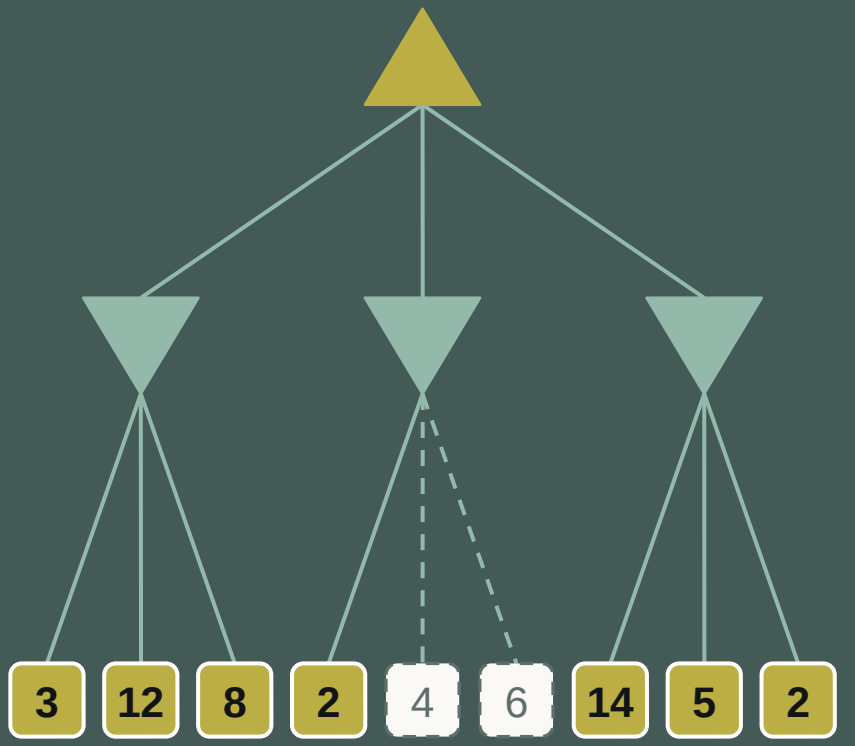
```
>>> minimax("root")
  leaf a11 = 3
  leaf a12 = 12
  leaf a13 = 8
  leaf a21 = 2
  leaf a22 = 4
  leaf a23 = 6
  leaf a31 = 14
  leaf a32 = 5
  leaf a33 = 2
(3, 'a1')
```



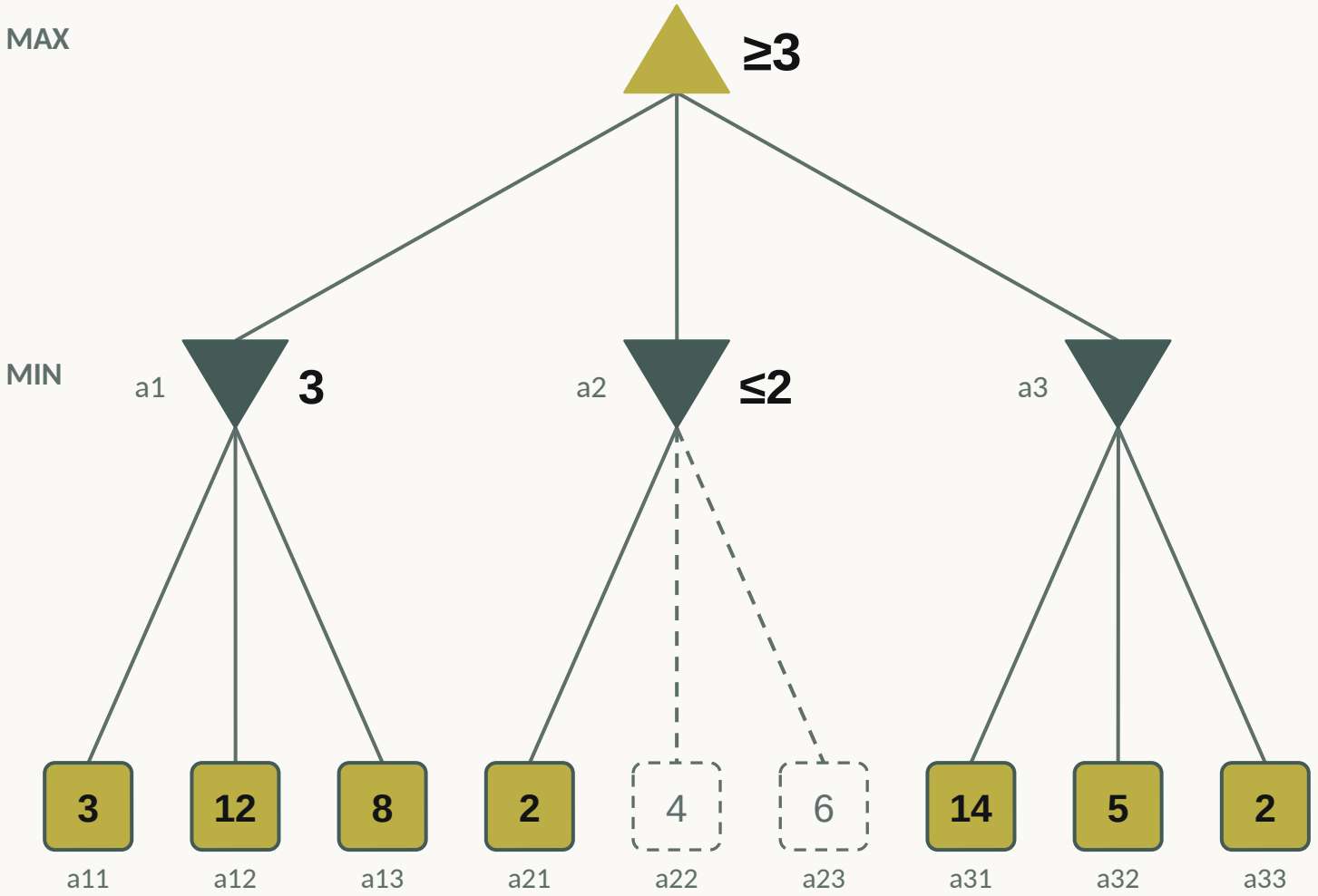
9 of 9 leaves evaluated. Minimax never skips anything.

Part 2: Alpha-beta

Same decision, fewer leaves



Stop when the answer can no longer change



1 **α: MAX's safe value**

The best MAX can already guarantee. After a1, α = 3.

2 **β: MIN's safe value**

The best MIN can already guarantee in the current branch.

3 **The cut at a2**

a21 = 2 means MIN can force ≤ 2 here. MAX already has 3, so a22 and a23 can't matter.

From pseudocode to Python

Pseudocode (course slides M6, slide 27)

```
function MAX-VALUE(state, α, β)
  if TERMINAL-TEST(state)
    then return UTILITY(state)
1   v ← −∞
2   for a, s in SUCCESSORS(state) do
3     v ← MAX(v, MIN-VALUE(s, α, β))
4     if v ≥ β then return v
5     α ← MAX(α, v)
6   return v
```

Python

```
def max_value(kids, alpha, beta):
1       v, best = -math.inf, None
2       for i, kid in enumerate(kids):
3           kid_v, _ = alphabeta(kid, alpha, beta)
            if kid_v > v:
                v, best = kid_v, kid
4           if v >= beta:                    # beta cut
                prune(kids[i + 1:])
                return v, best
5           alpha = max(alpha, v)
6       return v, best
```

Almost line for line. The TERMINAL-TEST lines live in alphabeta(). The only addition is best: Python returns the pair (v, best) so we also know which move produced v.

alphabeta(): deciding who moves

```
    def alphabeta_search(pos):
1       return alphabeta(pos, -math.inf, math.inf)

    def alphabeta(pos, alpha, beta):
2       kids = children(pos)
3       if not kids:                              # terminal
            return leaf_value(pos), None
4       if pos in MAX_TO_MOVE:
            return max_value(kids, alpha, beta)
5       return min_value(kids, alpha, beta)
```

1. **Widest window first**

   α = −∞ and β = +∞, with math.inf.

2. **Collect the children**

   Same helper as in minimax.

3. **Leaf: return its utility**

   This is TERMINAL-TEST.

4. **MAX node**

   max_value starts with v = −∞.

5. **MIN node**

   min_value starts with v = +∞.

min_value() is the mirror image

```
    def min_value(kids, alpha, beta):
1       v, best = math.inf, None
        for i, kid in enumerate(kids):
            kid_v, _ = alphabeta(kid, alpha, beta)
2           if kid_v < v:
                v, best = kid_v, kid
3           if v <= alpha:                    # alpha cut
                prune(kids[i + 1:])
                return v, best
4           beta = min(beta, v)
        return v, best

    def prune(rest):
        if rest:
            print(f"  prune: skip {rest}")
```

kids[i + 1:] are the siblings not yet explored; prune() just prints them.

1 Start at +∞

v = math.inf instead of -math.inf.

2 Keep the smallest value

kid_v < v instead of kid_v > v.

3 Alpha cut

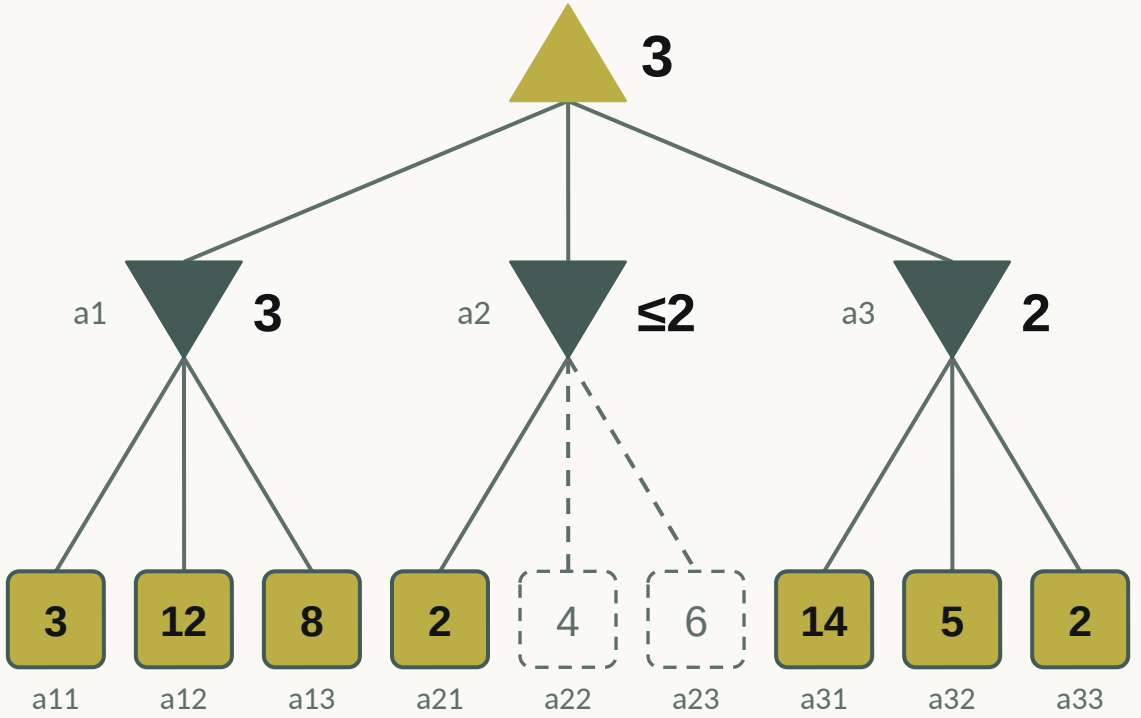
If v ≤ α, MAX already has something better elsewhere: stop.

4 Tighten β

β only goes down, and the next siblings see the new value.

Running alpha-beta

```
>>> alphabeta_search("root")
  leaf a11 = 3
  leaf a12 = 12
  leaf a13 = 8
  leaf a21 = 2
  prune: skip ['a22', 'a23']
  leaf a31 = 14
  leaf a32 = 5
  leaf a33 = 2
(3, 'a1')
```



7 of 9 leaves evaluated, and exactly the same answer: a1 with value 3.

Run it from the Python console

```
1  $ python3
2  >>> from minimax_alphabeta import *
3  >>> minimax("root")
     leaf a11 = 3
     leaf a12 = 12
     … one line per leaf …
     leaf a33 = 2
4  (3, 'a1')
   >>> minimax("a2")
     leaf a21 = 2
     leaf a22 = 4
     leaf a23 = 6
   (2, 'a21')
```

The leaf lines are not the answer: leaf_value() prints them while the search runs.

1 In the terminal

python3 opens the interactive console. The prompt changes to >>>

2 Load the file

import brings in TREE, UTILITY and every function.

3 Call the function

Give it the position you want to solve.

4 Python’s answer

The tuple (value, move). Any node works: at a2, MIN picks a21.

Predict first, then run

1 Change one leaf

Which leaves are pruned now? Does MAX still choose a1?

```
>>> UTILITY["a21"] = 4
>>> alphabeta_search("root")
```

2 Reorder the moves

How many leaves are evaluated now?

```
>>> UTILITY["a21"] = 2                    # undo exercise 1
>>> TREE["a3"] = ["a33", "a31", "a32"]
>>> alphabeta_search("root")
```

3 Start somewhere else

a22 and a23 were pruned before. Why not now?

```
>>> alphabeta_search("a2")
```

These lines change the tree only in this console session; the file stays the same.

Prolog and Python, side by side

Idea	Prolog	Python
The tree	`move(root, a1).`	`TREE["root"]`
Children	`findall(C, move(P, C), Cs)`	`children(pos)`
Loop over children	`recursion on [C|Cs]`	`for kid in kids:`
Best of the pairs	`max_member(Best, Pairs)`	`max(pairs, key=...)`
Infinity	`-1000 and 1000`	`math.inf`
Result	`Move = a1, Val = 3`	`(3, 'a1')`

Same algorithm, two styles: Prolog describes the tree as facts and searches it by recursion; Python stores it as data and walks it with loops.

Same answer, less work

Alpha-beta never changes the answer of minimax.
Good move ordering makes it prune even more.

9 leaves with minimax

7 with alpha-beta

5 with a33 explored first